

Documentation Index

Fetch the complete documentation index at: <https://docs.langchain.com/llms.txt>

Use this file to discover all available pages before exploring further.

LangSmith Studio

When building agents with LangChain locally, it's helpful to visualize what's happening inside your agent, interact with it in real-time, and debug issues as they occur. **LangSmith Studio** is a free visual interface for developing and testing your LangChain agents from your local machine.

Studio connects to your locally running agent to show you each step your agent takes: the prompts sent to the model, tool calls and their results, and the final output. You can test different inputs, inspect intermediate states, and iterate on your agent's behavior without additional code or deployment.

This page describes how to set up Studio with your local LangChain agent.

Prerequisites

Before you begin, ensure you have the following:

- **A LangSmith account:** Sign up (for free) or log in at smith.langchain.com.
- **A LangSmith API key:** Follow the [Create an API key](#) guide.
- If you don't want data traced to LangSmith, set `LANGSMITH_TRACING=false` in your application's `.env` file. With tracing disabled, no data leaves your local server.

Set up local Agent server

1. Install the LangGraph CLI

The LangGraph CLI provides a local development server (also called Agent Server) that connects your agent to Studio.

```
# Python >= 3.11 is required.
pip install --upgrade "langgraph-cli[inmem]"
```

2. Prepare your agent

If you already have a LangChain agent, you can use it directly. This example uses a simple email agent:

```
# agent.py
from langchain.agents import create_agent

def send_email(to: str, subject: str, body: str):
    """Send an email"""
    email = {
        "to": to,
        "subject": subject,
        "body": body
    }
    # ... email sending logic

    return f"Email sent to {to}"

agent = create_agent(
    "gpt-5.5",
    tools=[send_email],
    system_prompt="You are an email assistant. Always use the send_email tool.",
)
```

3. Environment variables

Studio requires a LangSmith API key to connect your local agent. Create a `.env` file in the root of your project and add your API key from [LangSmith](#).

Warning: Ensure your `.env` file is not committed to version control, such as Git.

```
# .env
LANGSMITH_API_KEY=lsv2...
```

4. Create a LangGraph config file

The LangGraph CLI uses a configuration file to locate your agent and manage dependencies. Create a `langgraph.json` file in your app's directory:

```
// langgraph.json
{
  "dependencies": ["."],
  "graphs": {
    "agent": "./src/agent.py:agent"
  },
  "env": ".env"
}
```

The `create_agent` function automatically returns a compiled LangGraph graph, which is what the `graphs` key expects in the configuration file.

For detailed explanations of each key in the JSON object of the configuration file, refer to the LangGraph configuration file reference ([/langsmith/cli#configuration-file](#)).

At this point, the project structure will look like this:

```
my-app/
├── src
│   └── agent.py
├── .env
└── langgraph.json
```

5. Install dependencies

Install your project dependencies from the root directory:

```
# pip
pip install langchain langchain-openai

# uv
uv add langchain langchain-openai
```

6. View your agent in Studio

Start the development server to connect your agent to Studio:

```
langgraph dev
```

Warning: Safari blocks `localhost` connections to Studio. To work around this, run the above command with `--tunnel` to access Studio via a secure tunnel.

Once the server is running, your agent is accessible both via API at `http://127.0.0.1:2024` and through the Studio UI at `https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024`:

[Screenshot: Agent view in the Studio UI]

Image could not be embedded automatically (source blocked the request).

View it directly at:

[studio_create-agent.png](#)

With Studio connected to your local agent, you can iterate quickly on your agent's behavior. Run a test input, inspect the full execution trace including prompts, tool arguments, return values, and token/latency metrics. When something goes wrong, Studio captures exceptions with the surrounding state to help you understand what happened.

The development server supports hot-reloading—make changes to prompts or tool signatures in your code, and Studio reflects them immediately. Re-run conversation threads from any step to test your changes without starting over. This workflow scales from simple single-tool agents to complex multi-node graphs.

For more information on how to run Studio, refer to the following guides in the LangSmith docs

([/langsmith/observability](#)):

- [Run application \(/langsmith/use-studio#run-application \)](#)
- [Manage assistants \(/langsmith/use-studio#manage-assistants \)](#)
- [Manage threads \(/langsmith/use-studio#manage-threads \)](#)
- [Iterate on prompts \(/langsmith/observability-studio \)](#)
- [Debug LangSmith traces \(/langsmith/observability-studio#debug-langsmith-traces \)](#)
- [Add node to dataset \(/langsmith/observability-studio#add-node-to-dataset \)](#)

Video guide

[Embedded video: "Studio"]

Watch on YouTube: <https://www.youtube.com/watch?v=Ml1gSIHwZLM>

Tip: For more information about deployed agents, see [Deployment \(/oss/python/langchain/deploy \)](#).

📖 [Connect these docs](#) to Claude, VSCode, and more via MCP for real-time answers.

⇒ [Edit this page on GitHub](#) or [file an issue](#).